

Module 6 — Evaluation & Observability

Scenario: make quality measurable, then make it enforceable.

Up to Module 5, TripPilot does a lot — RAG answers, tool calls, a governed booking workflow, memory, MCP. But nobody can say *how well*. Did a change improve answer quality or quietly break retrieval? We've been shipping on vibes. This module fixes that: you'll trace every run, put real numbers on retrieval and answer quality, assert the booking workflow's outcomes, and wire a CI gate so a regression can't merge silently.

The big picture

Piece	File(s)	What it proves
Tracing	@traceable in nodes / mcp_client / rag_service, run_graph.py	Every node, tool call, and LLM call is visible in LangSmith
Retrieval eval	evaluation/retrieval_eval.py, tests/test_eval_retrieval.py	The retriever finds the correct source doc (Precision@k / Recall@k / MRR)
Generation eval	evaluation/generation_eval.py, tests/test_eval_generation.py	Answers are faithful; hallucinated visa info is caught
Scenario eval	evaluation/scenario_eval.py, tests/test_eval_scenarios.py	The 10 booking conversations end in the right outcome
Report + CI	evaluation/generate_report.py, .github/workflows/eval.yml	Numbers are auto-generated; a regression fails the build

Prerequisites (one-time setup, if previously done please ignore)

```
git clone <your fork>
cd course_project-main
python -m venv .venv && .venv\Scripts\activate      # Windows
pip install -r requirements.txt

# Ollama is a separate runtime (not a pip package). Install from ollama.com, then:
ollama pull llama3.1:8b      # the app's answer model (MODEL_NAME)
ollama pull qwen2.5:7b      # the judge model (JUDGE_MODEL_NAME) -
deliberately different

cp .env.example .env      # then paste your free LangSmith key into .env
python seed/load_data.py  # build the SQLite data
python ingest.py          # build the Chroma vector DB
```

Your .env needs: MODEL_NAME, OLLAMA_BASE_URL, CHROMA_DB, DOCUMENT_PATH, LANGSMITH_TRACING=true, LANGSMITH_API_KEY, LANGSMITH_PROJECT, JUDGE_MODEL_NAME.

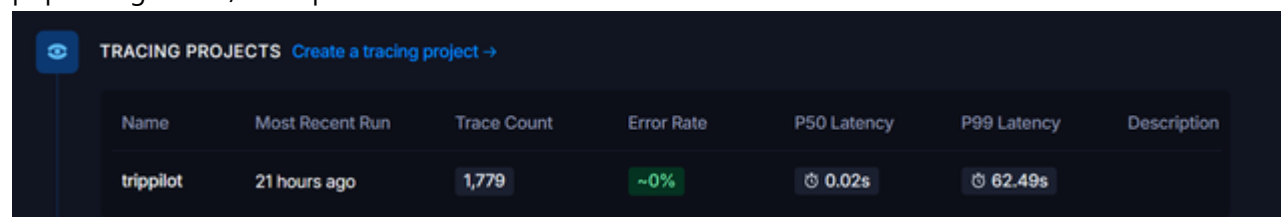
Step 1 — Trace the whole graph with LangSmith

You can't improve what you can't see. Tracing records every step of a run — each graph node, each MCP tool call, each LLM call — as a nested tree you can inspect, time, and compare in the LangSmith UI.

All 8 LangGraph nodes (`collect_requirements`, `load_traveler_context`, `search`, `assemble_itinerary`, `budget_check`, `present_options`, `execute_booking`, `confirm`), the MCP tool layer (`mcp_client.call_tool`), and the RAG retriever (`rag_service.retrieve_documents`) are decorated with `@traceable`. Setting `LANGSMITH_TRACING=true` in `.env` turns it on — no code changes needed to toggle it. Each booking conversation is grouped into one LangSmith **Thread** because `run_graph.py/resume_graph.py` pass the same `thread_id` in both `config["configurable"]` (checkpointing) and `config["metadata"]` (thread grouping).

```
# terminal 1
uvicorn mock_travel_api.main:app --port 8000
# terminal 2
python run_graph.py                # prints a "Thread ID: ..." then pauses for
approval
python resume_graph.py <thread_id> # paste that Thread ID
```

What you should see. In the `trippilot` project in LangSmith: one trace whose tree shows every node + tool span; a Threads tab grouping the run and the resume into one conversation; the prebuilt Dashboard populating Traces/Tools panels.



The screenshot shows the 'TRACING PROJECTS' section of the LangSmith dashboard. It features a table with columns for Name, Most Recent Run, Trace Count, Error Rate, P50 Latency, P99 Latency, and Description. The 'trippilot' project is listed with 1,779 traces, an error rate of ~0%, and latencies of 0.02s (P50) and 62.49s (P99).

Name	Most Recent Run	Trace Count	Error Rate	P50 Latency	P99 Latency	Description
trippilot	21 hours ago	1,779	~0%	0.02s	62.49s	

The screenshot displays the tripplilot interface. The top section shows a table of threads with columns: Turns, First Input, Last Output, Thread ID, Last Error, First Start Time, Last Start Time, P50 Latency, and P99 Latency. The bottom section shows a detailed view of a thread, including a summary of turns and a breakdown of the thread's components (Feedback, Input, Output) with their respective fields and values.

Turns	First Input	Last Output	Thread ID	Last Error	First Start Time	Last Start Time	P50 Latency	P99 Latency
4	Osaka	rejected	human_rejects_aft... slon		8/10/2026, 2:58:25 ...	8/10/2026, 3:24:30 ...	0.01s	0.02s
4	Dubai	approved	happy_path_multipl... ions		8/10/2026, 2:58:25 ...	8/10/2026, 3:24:30 ...	0.01s	0.01s
2	Bali	Trip exceeds budge...	escalated_after_tw... ions		8/10/2026, 2:58:25 ...	8/10/2026, 3:24:30 ...	0.01s	0.01s
4	Kyoto	approved	revise_once_then,... dget		8/10/2026, 2:58:25 ...	8/10/2026, 3:24:30 ...	0.02s	0.02s

The detailed view shows the following components:

- Summary:** 1 LangGraph
- Feedback:** (Empty)
- Input:**
 - Fields: destination Osaka, traveler_name Yuki, budget 500
- Output:**
 - Fields: destination Osaka, final_message Trip exceeds budget. Revision 1 of 2., idempotency_key 1fa43771-d658-42b2-9dfa-1000bc810fec, traveler_name Yuki, approval_feedback null, approval_status null

Step 2 — Retrieval evaluation (is it finding the right doc?)

RAG is only as good as its retrieval. If the retriever hands the LLM the wrong document, no amount of prompting saves the answer. So we measure it directly against a golden set.

`seed/eval/golden_qa.jsonl` holds 45 question → expected-source-doc → reference-answer triples drawn from the real `seed/docs/`. For each question, `retrieval_eval.py::compute_metrics()` retrieves the top-k chunks and checks whether the expected document's filename is among them, computing:

- **Recall@k** — did the right doc appear anywhere in the top k?
- **Precision@k** — what fraction of retrieved docs were the right one?
- **MRR** — how high was the right doc ranked (1.0 = always first)?

These are computed locally with no LLM and no network, so they can gate CI.

```
pytest tests/test_eval_retrieval.py -v
python -m evaluation.retrieval_eval # prints the raw metric dict
```

What you should see (real run, $k=4$): $\text{recall_at_k}=1.0$, $\text{mrr}=0.989$, $\text{precision_at_k}=0.25$. The 0.25 precision is *expected, not a bug*: each question has exactly one relevant doc, so precision@k maxes out at $1/k$. Recall and MRR are the signals that matter.

Comparison run (the "change one variable, re-run" deliverable). We re-ran across $k=1/4/8$:

k	recall_at_k	precision_at_k	mrr
1	0.978	0.978	0.978
4	1.000	0.250	0.989
8	1.000	0.250	0.989

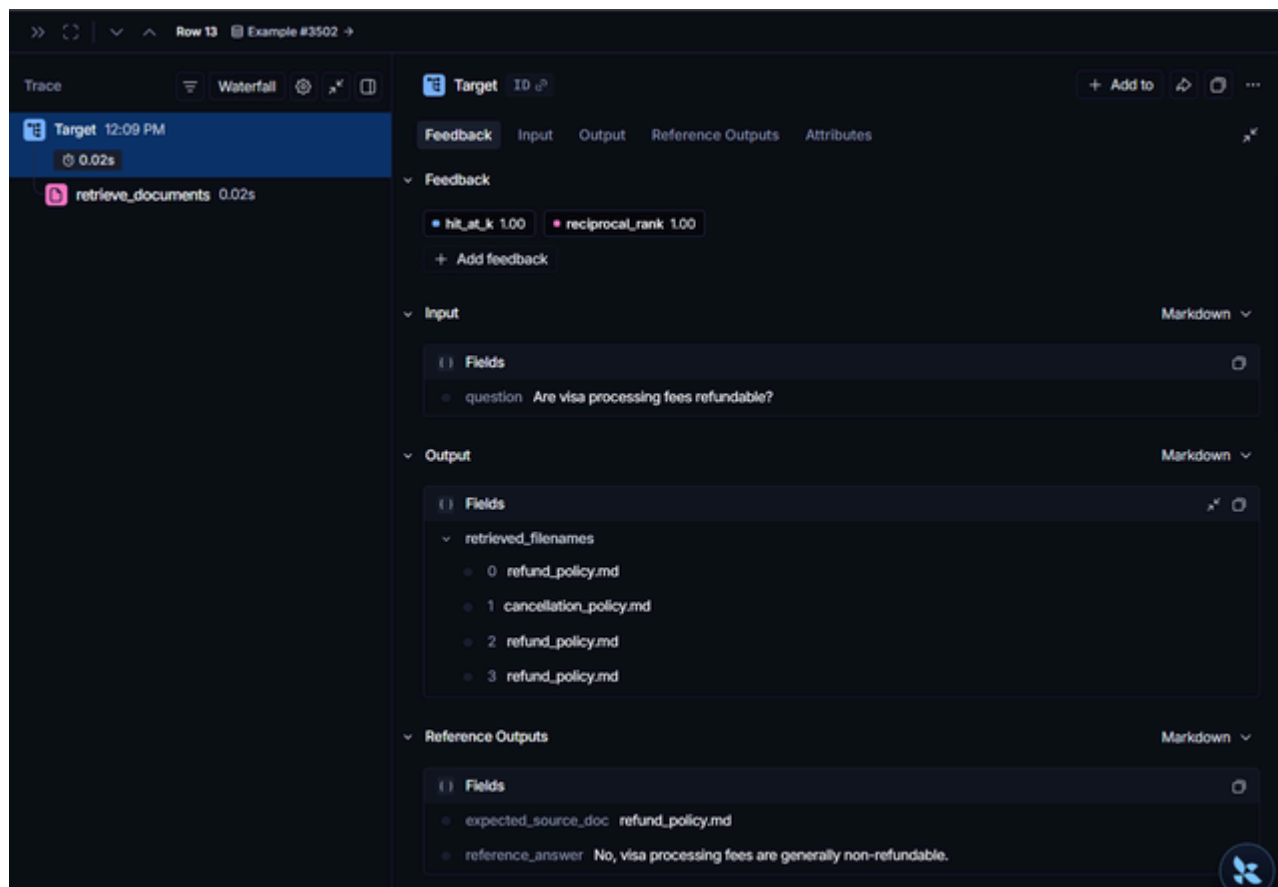
Reading the numbers: $k=1$ already gets the right doc *first* 97.8% of the time. Going to $k=4$ recovers the last 2.2% of recall (and nudges MRR up) for the few ambiguous questions — the precision "drop" is just the $1/k$ artifact, not a quality loss. $k=8$ is identical to $k=4$, so retrieving wider only adds noise and latency. **Conclusion: $k=4$ is the sweet spot, and that's what ships.**

```
===== test session starts =====
platform win32 -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\OzeanKhan\Documents\projects\fast_paced_course_project\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\OzeanKhan\Documents\projects\fast_paced_course_project
plugins: anyio-4.14.2, langsmith-0.10.15
collected 2 items

tests/test_eval_retrieval.py::test_retrieval_metrics_computed PASSED [ 50%]
tests/test_eval_retrieval.py::test_retrieval_quality_gate PASSED [100%]

===== 2 passed in 16.88s =====
(.venv) PS C:\Users\OzeanKhan\Documents\projects\fast_paced_course_project> python -m evaluation.retrieval_eval
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100% [103/103 [00:00<00:00, 6315.98it/s]]
{'k': 4, 'n_examples': 45, 'recall_at_k': 1.0, 'precision_at_k': 0.25, 'mrr': 0.9888888888888889}
View the evaluation results for experiment: 'retrieval-eval-9e5372e9' at:
https://smith.langchain.com/o/87a4b6ea-8864-4ea0-951c-d9ddf6037c41/datasets/132ca5dd-0ec9-40ac-9184-804d0456ef25/compare?selectedSessions=cfa562ec-0
ae8-4253-b3a5-22e65308f340
45it [00:01, 34.68it/s]
```

Personal / Datasets & Experiments / tripilot-golden-qa / retrieval-eval-edf993c7						
retrieval-eval-edf993c7 ID 45 / 45 runs fast_p.../00c3a9						
Inputs	Reference Outputs	Outputs	hit_at... 1.00 avg	recip... 0.99 avg	Latency 0.018 p50	Time
13 Are visa processing fees refundable?	refund_policy.md	["refund_policy.md","cancellation_policy.n	1.00	1.00	0.02s	
14 What currency is used in Tokyo?	tokyo.md	["tokyo.md","kyoto.md","osaka.md","kyoto	1.00	1.00	0.02s	
15 What passport validity is required for a J... pan tourist visa?	japan_visa.md	["japan_visa.md","japan_visa.md","japan_vi	1.00	1.00	0.02s	



Step 3 — Generation evaluation (are the answers faithful?)

Retrieval can be perfect and the model can still hallucinate. For a travel agent the flagship failure is a **wrong visa answer** — that strands a traveler at the airport. So we grade answer faithfulness, and prove the grader catches hallucinations.

the key discipline. We use **LLM-as-judge**, but with a twist that avoids the classic bias: the judge is a *different* local model (`JUDGE_MODEL_NAME`, e.g. `qwen2.5:7b`) than the app's answer model (`MODEL_NAME`, e.g. `llama3.1:8b`) — so the model isn't grading its own homework.

- `run_generation_eval()` — grades all 45 answers 1–5 for faithfulness against the reference; reports overall + visa-only averages.
- `hallucinated_visa_case()` — an explicit adversarial probe: it feeds the judge a deliberately wrong France-visa answer ("365 days", "no passport") and asserts the judge flags it. This is the domain's mandated failure case.
- `calibrate_judge()` — **don't trust a judge you haven't checked.** Before using it, we run it over `seed/eval/judge_calibration.jsonl`: 12 hand-labeled faithful / not-faithful answer pairs. The test asserts the judge agrees with the human labels $\geq 80\%$ of the time. This is the "build a judge you can trust" discipline — if the judge can't match obvious human calls, its 4.2/5 average means nothing.

Runtime note. This suite is intentionally long-running because it executes 58 sequential live LLM checks (45 golden-set cases + 12 judge-calibration cases + 1 adversarial hallucination case). On local Ollama hardware this can naturally take 40-50+ minutes. Run it before release/final QA and after major prompt, retrieval, or model changes; skip it during quick local iteration.

```
pytest tests/test_eval_generation.py -v
python -m evaluation.generation_eval
```

What you should see (real run): the judge agrees with the hand labels (calibration test passes), `avg_faithfulness_score` $\approx 4.2/5$, `avg_visa_faithfulness_score` $\approx 3.83/5$, and the hallucination case caught (`score=1`, `faithful=False`). Thresholds sit below the observed range so normal judge noise doesn't flake the gate, but a real regression (which drops scores to ~ 1 – 2) still trips it.

Step 4 — End-to-end scenario evaluation (does the workflow reach the right outcome?)

Individual pieces can pass while the whole conversation still goes wrong. So we run complete booking conversations and assert the final outcome.

`seed/eval/booking_scenarios.jsonl` holds 10 scripted conversations covering every branch: happy-path bookings, human rejections (incl. after a revision), budget escalation (one hitting the 2-revision cap), a revise-then-book case, and a 409-conflict-then-book case. `scenario_eval.py::run_all_scenarios()` runs each through the *real* graph, driving the approval interrupt with the scripted decision, and classifies the end state as `booked` / `rejected` / `escalated`.

`call_tool` is **mocked** here so the 10 outcomes are deterministic — a deliberate unit-test choice that isolates the graph's *decision logic*. The real MCP $\leftrightarrow$ HTTP stack is proven separately by `tests/test_langgraph.py::test_happy_path` (happy path, no mocks) and `tests/test_integration_booking.py` (a real-stack smoke test that hits the live `POST /bookings` for a ge 409 + idempotency replay).

How.

```
pytest tests/test_eval_scenarios.py -v
python -m evaluation.scenario_eval
```

What you should see: 10/10 scenarios match their expected outcome, including the conflict scenario re-presenting for a second approval after the 409.

Step 5 — Auto-generated report

A report you hand-edit drifts from reality. The metrics table should be produced *from the actual eval runs*, every time.

```
python -m evaluation.generate_report      # writes evaluation/EVAL_RESULTS.md
```

`generate_report.py` runs all three evals and writes `EVAL_RESULTS.md` purely from their return values — never edited by hand. That file is your metric-table + comparison-run deliverable.

Step 6 — CI quality gate + the deliberate regression

Measuring quality is only half the job; the point is to make it *enforceable* so a regression can't merge. Frame it like a dbt test: a data quality gate on model behavior.

`.github/workflows/eval.yml` runs on every push/PR. It builds the seed DB + Chroma and runs the **LLM-free** gates (retrieval + scenario evals). Those need no secrets, so your fork's CI runs green out of the box. Generation eval needs a local Ollama runtime, which GitHub-hosted runners don't have, so it stays a local-only gate.

prove the gate works (the deliberate regression).

1. On `main`, push — CI is **green**.
 2. On a branch, worsen something real — e.g. drop `k` to a value that hurts recall, or lower a threshold below the true score — and open a PR. CI goes **red**.
 3. Revert on `main` — green again.
-